

EFREI PARIS

Département de Mathématiques

Promotion 2028

Projet de Mathématiques pour le Machine Learning

Classification des chiffres manuscrits avec MNIST

Réalisé par :

Benjamin	BINISTI
Arthur	BOUTELIER
Thomas	BOTTALICO
Valentin	BELOUGNE
Baptiste	BERTRAND

1 Préparation et représentation des données

Par souci de simplicité, nous avons utilisé la base MNIST fournie par `tensorflow.keras.datasets`. Le dataset d'entraînement contient 60 000 images et celui de test en contient 10 000. Chaque image est de dimension 28×28 et représente un chiffre entre 0 et 9. Avant de passer à l'entraînement, nous avons vérifié l'intégrité des données. Nous avons également vérifié que les classes étaient équilibrées et correctes afin de ne pas fausser l'accuracy.

Les pixels sont initialement en niveaux de gris et ont une valeur comprise entre 0 et 255. On les normalise à l'aide d'une division par 255 pour éviter les effets d'échelle. On aplatit ensuite chaque image en un vecteur $x_i \in \mathbb{R}^{784}$. On obtient donc $X_{\text{train}} \in \mathbb{R}^{60000 \times 784}$ et $X_{\text{test}} \in \mathbb{R}^{10000 \times 784}$. On encode ensuite les labels au format one-hot, ce qui donne $Y_{\text{train}} \in \mathbb{R}^{60000 \times 10}$ et $Y_{\text{test}} \in \mathbb{R}^{10000 \times 10}$.

Afin d'optimiser notre code, nous manipulons les données, les poids et les biais sous forme de tableaux NumPy. Cela permet d'utiliser des produits matriciels plutôt que des boucles imbriquées. Par exemple, pour le modèle linéaire, les scores sont calculés en une seule opération : $O = XA^T + b$, avec $A \in \mathbb{R}^{10 \times 784}$, $b \in \mathbb{R}^{1 \times 10}$ et $O \in \mathbb{R}^{n \times 10}$. Nous utilisons la même logique pour les deux autres modèles.

2 Gradients utilisés pour l'apprentissage

Pour les trois modèles, nous utilisons softmax en sortie et l'entropie croisée comme fonction de coût. La log loss nous semble être le choix le plus naturel et adapté, puisqu'il s'agit d'un problème de classification multi-classe dont les sorties sont des probabilités alors que la MSE est par exemple plus adaptée à de la régression. Pour la couche de sortie, nous obtenons le résultat suivant :

$$\frac{\partial E_i}{\partial o_{i,k}} = P_{i,k} - y_i^{(k)}.$$

Ce terme correspond à l'erreur au niveau de la couche de sortie. Il est important car il nous permet de calculer le gradient des poids et des biais du modèle linéaire, ainsi que celui de la couche de sortie des modèles multicouches. Nous utilisons ensuite la règle de la chaîne et la proposition 5.2 du cours afin de rétropropager cette erreur dans les couches cachées. Les calculs détaillés sont donnés en annexe.

2.1 Modèle linéaire

Pour le modèle linéaire, les scores sont calculés directement à partir des pixels. On obtient donc :

$$\frac{\partial L}{\partial a_{k,m}} = \frac{1}{n} \sum_{i=1}^n (P_{i,k} - y_i^{(k)}) x_{i,m}, \quad \frac{\partial L}{\partial a_{k,0}} = \frac{1}{n} \sum_{i=1}^n (P_{i,k} - y_i^{(k)}).$$

2.2 MLP à une couche cachée

Pour le MLP à une couche cachée, nous reprenons les résultats du modèle linéaire pour la couche de sortie. Les gradients de cette couche sont donc identiques à ceux du modèle linéaire, avec $x_{i,m}$ remplacé par $z_{i,q}^1$. On utilise ensuite la proposition 5.2 du cours afin de déterminer le gradient de la couche cachée. On obtient :

$$\frac{\partial L}{\partial a_{q,m}^1} = \frac{1}{n} \sum_{i=1}^n x_{i,m} \varphi'(o_{i,q}^1) \left[\sum_{\ell=0}^9 a_{\ell,q}^2 (P_{i,\ell} - y_i^{(\ell)}) \right], \quad \frac{\partial L}{\partial a_{q,0}^1} = \frac{1}{n} \sum_{i=1}^n \varphi'(o_{i,q}^1) \left[\sum_{\ell=0}^9 a_{\ell,q}^2 (P_{i,\ell} - y_i^{(\ell)}) \right].$$

2.3 MLP à deux couches cachées

Pour le modèle à deux couches cachées, nous reprenons encore une fois les résultats précédents en les adaptant. La couche de sortie suit le même raisonnement, avec les sorties $z_{i,t}^2$ de la deuxième couche cachée. Pour la deuxième couche cachée, il suffit ensuite de remplacer les variables d'entrée par les

activations de la première couche. On obtient :

$$\frac{\partial L}{\partial a_{t,q}^2} = \frac{1}{n} \sum_{i=1}^n z_{i,q}^1 \varphi_2'(o_{i,t}^2) \left[\sum_{\ell=0}^9 a_{\ell,t}^3 (P_{i,\ell} - y_i^{(\ell)}) \right].$$

Enfin, nous calculons le gradient de la première couche cachée en rétropropageant l'erreur provenant des neurones de la deuxième couche cachée :

$$\frac{\partial L}{\partial a_{q,m}^1} = \frac{1}{n} \sum_{i=1}^n x_{i,m} \varphi_1'(o_{i,q}^1) \left[\sum_{t=1}^{p_2} a_{t,q}^2 \varphi_2'(o_{i,t}^2) \left(\sum_{\ell=0}^9 a_{\ell,t}^3 (P_{i,\ell} - y_i^{(\ell)}) \right) \right].$$

3 Choix techniques

Pour les couches cachées de nos modèles multicouches, nous avons choisi ReLU comme fonction d'activation. Nous avons voulu rester proches du cours tout en évitant les problèmes d'évanouissement du gradient que l'on peut rencontrer avec sigmoid ou tanh. L'utilisation de l'entropie croisée en fonction de coût ne suffit pas ici à éviter ce problème puisque la dérivée de la fonction d'activation apparaît encore dans le gradient des couches cachées. Avec ReLU, la dérivée vaut 1 pour les valeurs positives et 0 pour les valeurs négatives, ce qui facilite la rétropropagation dans les couches cachées.

Nous avons utilisé différentes initialisations selon les couches et les modèles. Pour le modèle linéaire, nous avons utilisé une initialisation aléatoire simple $0,01 \cdot \mathcal{N}(0, 1)$ afin d'éviter des poids identiques entre les classes et de garder des poids faibles au départ.

Pour les couches cachées des MLP, nous avons choisi une initialisation de type He de la forme $\mathcal{N}(0, 1) \sqrt{\frac{2}{n_{in}}}$, car elle est spécifiquement pensée pour ReLU et permet de garder des activations raisonnables. Enfin, pour les couches de sortie, nous avons utilisé une initialisation LeCun de la forme $\mathcal{N}(0, 1) \sqrt{\frac{1}{n_{in}}}$, plus modérée que He puisque la sortie est ici directement suivie d'un softmax.

On met ensuite les paramètres à jour à l'aide d'une descente de gradient batch, c'est-à-dire que nous recalculons le gradient sur l'ensemble du dataset d'entraînement à chaque epoch.

4 Résultats obtenus et choix des hyperparamètres

Afin de déterminer les meilleurs hyperparamètres comme le nombre de neurones par couche, le learning rate et le nombre d'epochs, nous avons implémenté une méthode de *Random Search*. Nous présentons ci-dessous les trois meilleurs résultats obtenus pour chaque type de modèle.

	Modèle	Trial	Training accuracy	Validation accuracy	Test accuracy	Final loss	Temps (s)	Learning rate	Epochs	Hidden dim	Hidden dim 1	Hidden dim 2
0	Linéaire	7	0.9205	0.9232	0.9215	0.2867	43.45	1.00	500	None	None	None
1	Linéaire	2	0.9204	0.9230	0.9218	0.2867	60.79	1.00	500	None	None	None
2	Linéaire	1	0.9205	0.9229	0.9215	0.2867	83.68	1.00	500	None	None	None
3	MLP 1 couche	1	0.9313	0.9357	0.9324	0.2475	368.32	0.20	350	512	None	None
4	MLP 1 couche	3	0.9315	0.9353	0.9336	0.2461	390.05	0.20	350	512	None	None
5	MLP 1 couche	5	0.9300	0.9342	0.9314	0.2524	267.13	0.20	350	256	None	None
6	MLP 2 couches	5	0.9520	0.9560	0.9516	0.1686	579.22	0.20	350	None	512	128
7	MLP 2 couches	2	0.9534	0.9555	0.9504	0.1654	840.85	0.20	350	None	512	256
8	MLP 2 couches	5	0.9507	0.9529	0.9485	0.1745	714.53	0.18	350	None	512	256

FIGURE 1 – Tableau des performances des meilleurs modèles sur MNIST.

On observe une progression avec la profondeur du réseau : le MLP à deux couches cachées atteint **95,16%** d'accuracy sur le test set, contre 92,18% pour le modèle linéaire, soit un gain d'environ 3%. Cette amélioration s'accompagne d'un coût en temps de calcul significativement plus élevé (environ 13

fois supérieur). Le modèle à une couche se situe entre les deux avec 93,36%. De plus, les modèles ne semblent pas overfitter puisque les différentes accuracies sont très proches les unes des autres.

En effectuant nos différents tests, nous avons remarqué qu'augmenter le learning rate et le nombre d'epochs permettait d'améliorer les performances, alors que l'augmentation du nombre de neurones par couche avait un impact bien plus marginal tout en faisant croître fortement le nombre de paramètres et les temps de calcul. Nous avons conclu que 512 neurones dans la première couche cachée (MLP1 et MLP2) et 128 neurones dans la deuxième couche (MLP2) constituaient un bon compromis. Le fait que le meilleur MLP2 n'utilise que 128 neurones dans sa deuxième couche confirme que l'augmenter davantage n'était pas utile. Il est également à noter que pour le modèle linéaire et le MLP2, l'utilisation d'un learning rate élevé a causé une légère instabilité, avec des sauts au cours de la descente de gradient. Nous avons donc choisi de ne pas l'augmenter davantage.

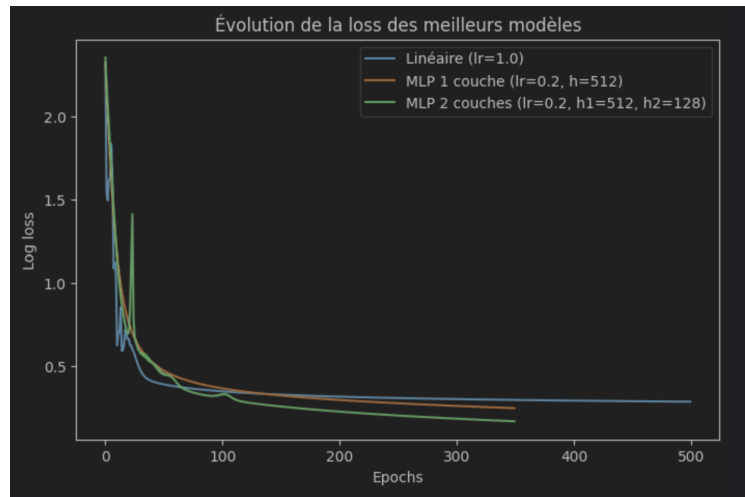


FIGURE 2 – Évolution de la log loss des meilleurs modèles au cours des epochs.

5 Analyse des erreurs

Malgré les bonnes performances globales, nos modèles commettent encore environ **4 à 8%** d'erreurs sur l'ensemble de test. Pour analyser ces erreurs, nous utilisons une matrice de confusion et une ACP.

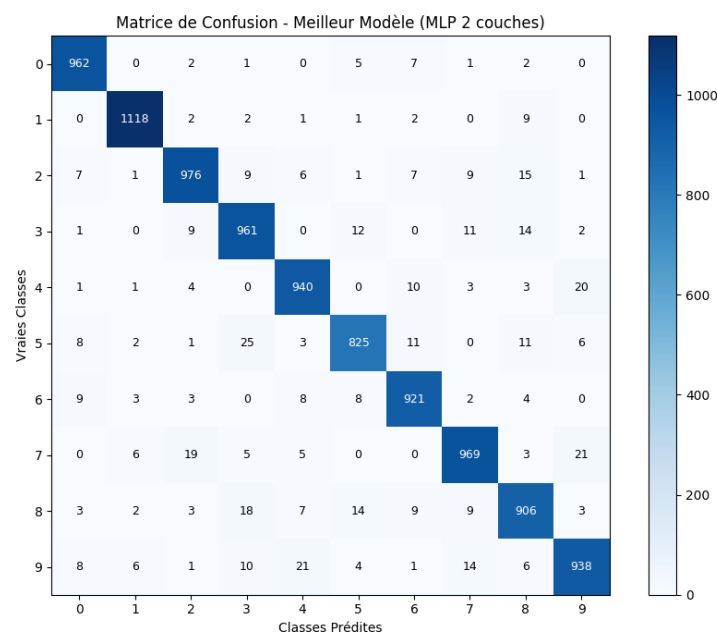


FIGURE 3 – Matrice de confusion du meilleur modèle MLP (2 couches cachées).

5.1 Étude des chiffres mal classés

L'analyse de la matrice de confusion et des échantillons d'erreurs nous permet de tirer les conclusions suivantes :

- **Les chiffres ambigus** : Une partie importante des erreurs concerne des chiffres aux formes assez proches. On observe notamment des confusions entre le **4 et le 9** (20 exemples du 4 classés comme 9, 21 exemples du 9 classés comme 4), ainsi qu'entre le **3 et le 5** (12 exemples du 3 classés comme 5, 25 exemples du 5 classés comme 3). Ces chiffres peuvent partager des structures visuelles similaires, ce qui induit parfois le réseau en erreur.
- **Les écritures atypiques** : Le modèle peine avec les styles d'écriture peu communs. Un chiffre trop penché, très épais ou mal centré dans le cadre de 28×28 pixels modifie la position des données et induit le réseau en erreur.
- **Limitations architecturales** : L'aplatissement de l'image 28×28 en un vecteur de 784 pixels ne permet pas au modèle de prendre directement en compte la structure spatiale. Il ne sait pas explicitement quels pixels sont voisins. Dès lors, si un chiffre est décalé ou dessiné à une échelle différente, les pixels ne tombent plus aux mêmes positions et le modèle perd ses repères.

Des exemples d'images mal classées ainsi que l'ACP des activations du meilleur modèle sont présentés en annexe afin de compléter cette analyse.

6 Conclusion

Cette première partie nous a permis de mettre en place une chaîne complète de classification sur MNIST, de la préparation des données jusqu'à l'analyse des erreurs. Nous avons implémenté trois architectures de complexité croissante – un modèle linéaire, un MLP à une couche cachée et un MLP à deux couches cachées – et avons montré qu'une profondeur accrue améliore les performances dans nos essais, avec un gain total de l'ordre de 3 points entre le modèle linéaire et le MLP à deux couches. Cette amélioration a cependant un coût en temps de calcul significatif.

6.1 Limitations

Un modèle linéaire ou même un réseau à une ou deux couches cachées reste limité pour traiter MNIST. Un modèle linéaire ne capture que des relations simples entre pixels et modélise difficilement les structures spatiales des chiffres (contours, formes). L'ajout de couches cachées améliore la capacité d'approximation, mais ces réseaux entièrement connectés ne prennent pas vraiment en compte la structure 2D de l'image. Ils apprennent donc moins efficacement que des architectures convolutionnelles et deviennent rapidement coûteux en paramètres, avec un risque de surapprentissage si l'architecture devient trop grande. De plus, leur profondeur limitée restreint leur capacité à extraire des caractéristiques hiérarchiques complexes.

Enfin, le dataset MNIST lui-même présente des limites : il est relativement simple, centré et peu bruité, ce qui ne reflète pas des conditions réelles. Sa faible diversité de style peut conduire à des modèles qui généralisent mal à des données plus complexes. L'utilisation de réseaux de neurones convolutionnels (CNN) constituerait une piste naturelle pour dépasser ces limitations et exploiter la structure spatiale des images.

Références

- Initialisation des poids :
<https://cs231n.github.io/neural-networks-2/>
- Log loss et problème de $\ln(0)$:
https://scikit-learn.org/stable/modules/generated/sklearn.metrics.log_loss.html
- Dérivée de softmax :
<https://mldawn.com/the-derivative-of-softmax-function-w-r-t-z/>
- Initialisation de LeCun :
<https://littlebit.ai/lecun-initialization-applicability-in-neural-network/>
- Initialisation des poids, source générale :
https://en.wikipedia.org/wiki/Weight_initialization

Annexes

Sommaire des annexes

- **Annexe A** : Calcul détaillé du gradient pour le modèle linéaire.
- **Annexe B** : Calcul détaillé du gradient pour le MLP à une couche cachée.
- **Annexe C** : Calcul détaillé du gradient pour le MLP à deux couches cachées.
- **Annexe D** : Analyse complémentaire des erreurs.
- **Annexe E** : Code du projet et dépôt GitHub.

A Calcul détaillé du gradient pour le modèle linéaire

Notations

On dispose de n exemples. Pour l'exemple i , on note :

- $x_i \in \mathbb{R}^{784}$ le vecteur d'entrée (pixels normalisés),
- $y_i \in \{0, 1\}^{10}$ le vecteur one-hot associé au label de x_i ,
- $A \in \mathbb{R}^{10 \times 784}$ la matrice de poids, $b \in \mathbb{R}^{1 \times 10}$ le vecteur de biais,
- $o_{i,k} = \sum_{m=1}^{784} a_{k,m} x_{i,m} + a_{k,0}$ le score de la classe k pour l'exemple i ,
- $P_{i,k} = \frac{e^{o_{i,k}}}{\sum_{j=0}^9 e^{o_{i,j}}}$ la probabilité softmax associée,
- $E_i = -\sum_{k=0}^9 y_i^{(k)} \log P_{i,k}$ la perte d'entropie croisée pour l'exemple i ,
- $L = \frac{1}{n} \sum_{i=1}^n E_i$ la perte moyenne sur le batch.

Étape 1 : gradient de la perte par rapport aux scores de sortie

On calcule $\frac{\partial E_i}{\partial o_{i,k}}$. Par la règle de la chaîne :

$$\frac{\partial E_i}{\partial o_{i,k}} = -\sum_{j=0}^9 y_i^{(j)} \frac{\partial \log P_{i,j}}{\partial o_{i,k}} = -\sum_{j=0}^9 y_i^{(j)} \frac{1}{P_{i,j}} \frac{\partial P_{i,j}}{\partial o_{i,k}}.$$

On calcule la dérivée du softmax. Pour $j \neq k$:

$$\frac{\partial P_{i,j}}{\partial o_{i,k}} = \frac{-e^{o_{i,j}} e^{o_{i,k}}}{(\sum_{\ell} e^{o_{i,\ell}})^2} = -P_{i,j} P_{i,k}.$$

Pour $j = k$:

$$\frac{\partial P_{i,k}}{\partial o_{i,k}} = \frac{e^{o_{i,k}} \sum_{\ell} e^{o_{i,\ell}} - e^{2o_{i,k}}}{(\sum_{\ell} e^{o_{i,\ell}})^2} = P_{i,k}(1 - P_{i,k}).$$

On peut écrire de façon unifiée :

$$\frac{\partial P_{i,j}}{\partial o_{i,k}} = P_{i,j}(\delta_{jk} - P_{i,k}),$$

où δ_{jk} est le symbole de Kronecker. On en déduit :

$$\frac{\partial \log P_{i,j}}{\partial o_{i,k}} = \frac{1}{P_{i,j}} P_{i,j}(\delta_{jk} - P_{i,k}) = \delta_{jk} - P_{i,k}.$$

En substituant dans l'expression de $\frac{\partial E_i}{\partial o_{i,k}}$:

$$\frac{\partial E_i}{\partial o_{i,k}} = -\sum_{j=0}^9 y_i^{(j)} (\delta_{jk} - P_{i,k}) = -y_i^{(k)} + P_{i,k} \sum_{j=0}^9 y_i^{(j)}.$$

Comme y_i est un vecteur one-hot, $\sum_{j=0}^9 y_i^{(j)} = 1$. On obtient donc :

$$\boxed{\frac{\partial E_i}{\partial o_{i,k}} = P_{i,k} - y_i^{(k)}}.$$

Étape 2 : gradient par rapport aux poids

Pour le modèle linéaire, $o_{i,k}$ ne dépend de $a_{k',m}$ que si $k' = k$. Par la règle de la chaîne :

$$\frac{\partial E_i}{\partial a_{k,m}} = \frac{\partial E_i}{\partial o_{i,k}} \cdot \frac{\partial o_{i,k}}{\partial a_{k,m}} = (P_{i,k} - y_i^{(k)}) \cdot x_{i,m}.$$

En moyennant sur le batch :

$$\frac{\partial L}{\partial a_{k,m}} = \frac{1}{n} \sum_{i=1}^n (P_{i,k} - y_i^{(k)}) x_{i,m}.$$

Étape 3 : gradient par rapport aux biais

Le biais $a_{k,0}$ joue le rôle d'une entrée constante égale à 1 :

$$\frac{\partial o_{i,k}}{\partial a_{k,0}} = 1.$$

On obtient de façon identique :

$$\frac{\partial L}{\partial a_{k,0}} = \frac{1}{n} \sum_{i=1}^n (P_{i,k} - y_i^{(k)}).$$

B Calcul détaillé du gradient pour le MLP à une couche cachée

Notations et indices utilisés

Pour déterminer le gradient de notre réseau de neurones à une couche cachée, on choisit d'utiliser les notations suivantes.

On considère le vecteur de l'image i aplatie :

$$x_i = (x_{i,1}, x_{i,2}, \dots, x_{i,784})$$

Chaque composante $x_{i,m}$ du vecteur représente ainsi un pixel de l'image i .

On choisit aussi d'utiliser les indices suivants :

$$i \in \{1, \dots, n\}$$

pour désigner une image

$$m \in \{1, \dots, 784\}$$

pour désigner un pixel fixé

$$r \in \{1, \dots, 784\}$$

comme indice muet dans les sommes sur les pixels

$$q \in \{1, \dots, p_1\}$$

pour désigner un neurone de la couche cachée fixé

$$s \in \{1, \dots, p_1\}$$

comme indice muet dans les sommes sur les neurones de la couche cachée

$$k \in \{0, \dots, 9\}$$

pour désigner une classe quelconque fixée

$$\ell \in \{0, \dots, 9\}$$

comme indice muet dans les sommes sur les classes.

Le nombre de neurones dans la couche cachée est noté :

$$p_1.$$

Pour une image i , le label est encodé sous forme one-hot :

$$y_i^{(k)} = \begin{cases} 1 & \text{si l'image } i \text{ appartient à la classe } k, \\ 0 & \text{sinon.} \end{cases}$$

Propagation du modèle à une couche cachée

On utilise les formules suivantes pour chaque neurone de la couche cachée $q \in \{1, \dots, p_1\}$:

$$o_{i,q}^1 = \sum_{r=1}^{784} a_{q,r}^1 x_{i,r} + a_{q,0}^1.$$

On utilise r comme indice muet pour parcourir tous les pixels. Le terme $a_{q,r}^1$ est le poids reliant le pixel r au neurone caché q et $a_{q,0}^1$ est le biais de ce neurone.

On calcule ensuite la sortie de ce neurone en utilisant la fonction d'activation ϕ :

$$z_{i,q}^1 = \phi(o_{i,q}^1).$$

La couche de sortie calcule ensuite, pour chaque classe $k \in \{0, \dots, 9\}$:

$$o_{i,k} = \sum_{s=1}^{p_1} a_{k,s}^2 z_{i,s}^1 + a_{k,0}^2.$$

On utilise s comme indice muet pour parcourir tous les neurones de la couche cachée. Le terme $a_{k,s}^2$ est le poids reliant le neurone s de la couche cachée à celui représentant la classe k de la couche de sortie et $a_{k,0}^2$ est le biais associé au neurone k de la couche de sortie.

On utilise ensuite la fonction softmax comme fonction d'activation de la couche de sortie afin d'obtenir des probabilités :

$$P_{i,k} = \frac{e^{o_{i,k}}}{\sum_{\ell=0}^9 e^{o_{i,\ell}}}.$$

On utilise aussi comme fonction de coût l'entropie croisée pour une image i :

$$E_i = - \sum_{\ell=0}^9 y_i^{(\ell)} \ln(P_{i,\ell}).$$

Donc la fonction coût totale est alors la moyenne sur toutes les images :

$$L = \frac{1}{n} \sum_{i=1}^n E_i.$$

Gradient des poids de la couche de sortie

On cherche ensuite à calculer le gradient des poids et biais pour les différentes couches. On commence pour cela à calculer les gradients de la couche de sortie.

Pour cela, on réutilise les résultats obtenus pour le modèle linéaire mais on effectue quelques modifications. En effet, les scores étaient calculés directement à partir des pixels :

$$o_{i,k} = \sum_{m=1}^{784} a_{k,m} x_{i,m} + a_{k,0}.$$

Et on avait alors obtenu les résultats suivants :

$$\frac{\partial L}{\partial a_{k,m}} = -\frac{1}{n} \sum_{i=1}^n x_{i,m} \left(y_i^{(k)} - P_{i,k} \right),$$

$$\frac{\partial L}{\partial a_{k,0}} = -\frac{1}{n} \sum_{i=1}^n \left(y_i^{(k)} - P_{i,k} \right).$$

Dans notre nouveau modèle avec une couche cachée, la couche de sortie a la même structure, mais au lieu de recevoir directement les pixels $x_{i,m}$, elle reçoit les sorties de la couche cachée $z_{i,q}^1$.

On calcule donc le score de la classe k de la façon suivante :

$$o_{i,k} = \sum_{q=1}^{p_1} a_{k,q}^2 z_{i,q}^1 + a_{k,0}^2.$$

On peut donc reprendre le raisonnement du modèle linéaire en remplaçant simplement les entrées $x_{i,m}$ par les activations cachées $z_{i,q}^1$.

Ainsi on obtient les résultats suivants pour les poids :

$$\frac{\partial L}{\partial a_{k,q}^2} = -\frac{1}{n} \sum_{i=1}^n z_{i,q}^1 \left(y_i^{(k)} - P_{i,k} \right)$$

De la même manière, on obtient les biais suivants, comme un biais peut être vu comme un poids relié à une entrée constante égale à 1 :

$$\frac{\partial L}{\partial a_{k,0}^2} = -\frac{1}{n} \sum_{i=1}^n \left(y_i^{(k)} - P_{i,k} \right)$$

Erreur au niveau de la couche de sortie

On utilise ensuite la proposition 5.2 du cours pour calculer le gradient de la couche cachée. Pour cela on a besoin de calculer le terme suivant $\frac{\partial E_i}{\partial o_{i,c}}$ où $c \in \{0, \dots, 9\}$ est une classe fixée.

On part de :

$$E_i = - \sum_{\ell=0}^9 y_i^{(\ell)} \ln(P_{i,\ell}).$$

En dérivant par rapport au score fixé $o_{i,c}$, on obtient :

$$\frac{\partial E_i}{\partial o_{i,c}} = - \sum_{\ell=0}^9 y_i^{(\ell)} \frac{\partial \ln(P_{i,\ell})}{\partial o_{i,c}}.$$

Or, en dérivant la fonction $\ln$, on obtient :

$$\frac{\partial E_i}{\partial o_{i,c}} = - \sum_{\ell=0}^9 y_i^{(\ell)} \frac{1}{P_{i,\ell}} \frac{\partial P_{i,\ell}}{\partial o_{i,c}}.$$

On utilise maintenant la définition de la fonction softmax :

$$P_{i,\ell} = \frac{e^{o_{i,\ell}}}{\sum_{s=0}^9 e^{o_{i,s}}}.$$

On peut alors faire une disjonction de cas de façon similaire à ce que l'on a utilisé pour notre premier modèle linéaire.

Premier cas : $\ell = c$

On a :

$$P_{i,c} = \frac{e^{o_{i,c}}}{\sum_{s=0}^9 e^{o_{i,s}}}.$$

Dans ce cas, le score $o_{i,c}$ apparaît à la fois au numérateur et au dénominateur. On obtient donc la dérivée suivante :

$$\frac{\partial P_{i,c}}{\partial o_{i,c}} = P_{i,c}(1 - P_{i,c}).$$

Deuxième cas : $\ell \neq c$

On a :

$$P_{i,\ell} = \frac{e^{o_{i,\ell}}}{\sum_{s=0}^9 e^{o_{i,s}}}.$$

Ici, seul le dénominateur dépend de $o_{i,c}$. On obtient donc :

$$\frac{\partial P_{i,\ell}}{\partial o_{i,c}} = -P_{i,\ell}P_{i,c}.$$

On remplace maintenant ces résultats dans la dérivée de E_i .
 Pour le terme où $\ell = c$, on obtient :

$$-y_i^{(c)} \frac{1}{P_{i,c}} P_{i,c} (1 - P_{i,c}) = -y_i^{(c)} (1 - P_{i,c}).$$

Pour les termes où $\ell \neq c$, on obtient :

$$-y_i^{(\ell)} \frac{1}{P_{i,\ell}} (-P_{i,\ell} P_{i,c}) = y_i^{(\ell)} P_{i,c}.$$

Ainsi :

$$\frac{\partial E_i}{\partial o_{i,c}} = -y_i^{(c)} (1 - P_{i,c}) + P_{i,c} \sum_{\ell \neq c} y_i^{(\ell)}.$$

D'où, en développant, on obtient :

$$\frac{\partial E_i}{\partial o_{i,c}} = -y_i^{(c)} + y_i^{(c)} P_{i,c} + P_{i,c} \sum_{\ell \neq c} y_i^{(\ell)}.$$

Et en factorisant par $P_{i,c}$, on obtient :

$$\frac{\partial E_i}{\partial o_{i,c}} = -y_i^{(c)} + P_{i,c} \left(y_i^{(c)} + \sum_{\ell \neq c} y_i^{(\ell)} \right).$$

Or, on sait que y_i est un vecteur one-hot, d'où :

$$y_i^{(c)} + \sum_{\ell \neq c} y_i^{(\ell)} = \sum_{\ell=0}^9 y_i^{(\ell)} = 1.$$

On obtient donc :

$$\boxed{\frac{\partial E_i}{\partial o_{i,c}} = P_{i,c} - y_i^{(c)}}$$

Ainsi, pour une classe quelconque ℓ , on peut écrire :

$$\boxed{\frac{\partial E_i}{\partial o_{i,\ell}} = P_{i,\ell} - y_i^{(\ell)}}$$

Gradient des poids de la couche cachée

On peut maintenant trouver le gradient par rapport à un poids de la première couche cachée, où q est un neurone caché fixé et m un pixel fixé :

$$\frac{\partial E_i}{\partial a_{q,m}^1}.$$

On utilise la formule de rétropropagation de la proposition 5.2 du cours :

$$\frac{\partial E_i}{\partial a_{q,k}^h} = \left(\sum_{j=1}^{p_{h+1}} \frac{\partial E_i}{\partial o_{i,j}^{h+1}} a_{j,q}^{h+1} \right) \phi'_h(o_{i,q}^h) z_{i,k}^{h-1}.$$

Dans notre cas, on étudie la première couche cachée ($h = 1$) et on étudie le poids $a_{q,m}^1$.
 La couche précédente est la couche d'entrée. On a donc :

$$z_{i,m}^0 = x_{i,m}.$$

La couche suivante est la couche de sortie, donc on fait la somme sur les classes :

$$\ell = 0, \dots, 9.$$

La formule de rétropropagation devient alors :

$$\frac{\partial E_i}{\partial a_{q,m}^1} = \left(\sum_{\ell=0}^9 \frac{\partial E_i}{\partial o_{i,\ell}} a_{\ell,q}^2 \right) \phi'(o_{i,q}^1) x_{i,m}.$$

On utilise maintenant le résultat obtenu précédemment :

$$\frac{\partial E_i}{\partial o_{i,\ell}} = P_{i,\ell} - y_i^{(\ell)}.$$

Et en remplaçant, on obtient :

$$\boxed{\frac{\partial E_i}{\partial a_{q,m}^1} = x_{i,m} \phi'(o_{i,q}^1) \left[\sum_{\ell=0}^9 a_{\ell,q}^2 (P_{i,\ell} - y_i^{(\ell)}) \right]}$$

Cette expression donne le gradient pour une seule image i et on veut maintenant calculer le coût moyen.

Passage au coût moyen

On sait que le coût moyen total est :

$$L = \frac{1}{n} \sum_{i=1}^n E_i.$$

Donc :

$$\frac{\partial L}{\partial a_{q,m}^1} = \frac{1}{n} \sum_{i=1}^n \frac{\partial E_i}{\partial a_{q,m}^1}.$$

D'où, en remplaçant, on obtient :

$$\boxed{\frac{\partial L}{\partial a_{q,m}^1} = -\frac{1}{n} \sum_{i=1}^n x_{i,m} \phi'(o_{i,q}^1) \left[\sum_{\ell=0}^9 a_{\ell,q}^2 (y_i^{(\ell)} - P_{i,\ell}) \right]}$$

Gradient du biais de la couche cachée

Pour le biais $a_{q,0}^1$, le raisonnement est le même. La seule différence est que le biais peut être vu comme un poids relié à une entrée constante égale à 1.

Ainsi, on remplace simplement $x_{i,m}$ par 1 dans la formule précédente.

Pour une image i :

$$\boxed{\frac{\partial E_i}{\partial a_{q,0}^1} = \phi'(o_{i,q}^1) \left[\sum_{\ell=0}^9 a_{\ell,q}^2 (P_{i,\ell} - y_i^{(\ell)}) \right]}$$

Et pour la fonction coût moyenne :

$$\boxed{\frac{\partial L}{\partial a_{q,0}^1} = -\frac{1}{n} \sum_{i=1}^n \phi'(o_{i,q}^1) \left[\sum_{\ell=0}^9 a_{\ell,q}^2 (y_i^{(\ell)} - P_{i,\ell}) \right]}$$

C Calcul détaillé du gradient pour le MLP à deux couches cachées

Notations et indices utilisés

Pour déterminer le gradient de notre réseau de neurones à deux couches cachées, on réutilise en grande partie les notations utilisées pour le modèle linéaire et pour le modèle à une couche cachée.

On considère le vecteur de l'image i aplatie :

$$x_i = (x_{i,1}, x_{i,2}, \dots, x_{i,784}).$$

Chaque composante $x_{i,m}$ du vecteur représente donc un pixel de l'image i .

On choisit aussi d'utiliser les indices suivants :

$$i \in \{1, \dots, n\}$$

pour désigner une image,

$$m \in \{1, \dots, 784\}$$

pour désigner un pixel fixé,

$$r \in \{1, \dots, 784\}$$

comme indice muet dans les sommes sur les pixels,

$$q \in \{1, \dots, p_1\}$$

pour désigner un neurone fixé de la première couche cachée,

$$s \in \{1, \dots, p_1\}$$

comme indice muet dans les sommes sur les neurones de la première couche cachée,

$$t \in \{1, \dots, p_2\}$$

pour désigner un neurone fixé de la deuxième couche cachée,

$$u \in \{1, \dots, p_2\}$$

comme indice muet dans les sommes sur les neurones de la deuxième couche cachée,

$$k \in \{0, \dots, 9\}$$

pour désigner une classe quelconque fixée,

$$\ell \in \{0, \dots, 9\}$$

comme indice muet dans les sommes sur les classes.

Le nombre de neurones dans la première couche cachée est noté :

$$p_1.$$

Le nombre de neurones dans la deuxième couche cachée est noté :

$$p_2.$$

Pour une image i , le label est encodé sous forme one-hot :

$$y_i^{(k)} = \begin{cases} 1 & \text{si l'image } i \text{ appartient à la classe } k, \\ 0 & \text{sinon.} \end{cases}$$

Propagation du modèle à deux couches cachées

On ajoute maintenant une deuxième couche cachée au modèle précédent.

La première couche cachée calcule, pour chaque neurone $q \in \{1, \dots, p_1\}$:

$$o_{i,q}^1 = \sum_{r=1}^{784} a_{q,r}^1 x_{i,r} + a_{q,0}^1.$$

On utilise r comme indice muet pour parcourir tous les pixels. Le terme $a_{q,r}^1$ est le poids reliant le pixel r au neurone q de la première couche cachée, et $a_{q,0}^1$ est le biais de ce neurone.

On calcule ensuite la sortie de ce neurone avec la fonction d'activation ϕ_1 :

$$z_{i,q}^1 = \phi_1(o_{i,q}^1).$$

La deuxième couche cachée reçoit les sorties $z_{i,q}^1$ de la première couche cachée. Pour chaque neurone $t \in \{1, \dots, p_2\}$, elle calcule :

$$o_{i,t}^2 = \sum_{s=1}^{p_1} a_{t,s}^2 z_{i,s}^1 + a_{t,0}^2.$$

On utilise s comme indice muet pour parcourir tous les neurones de la première couche cachée. Le terme $a_{t,s}^2$ est le poids reliant le neurone s de la première couche cachée au neurone t de la deuxième couche cachée, et $a_{t,0}^2$ est le biais associé à ce neurone.

On calcule ensuite la sortie du neurone t de la deuxième couche cachée :

$$z_{i,t}^2 = \phi_2(o_{i,t}^2).$$

Enfin, la couche de sortie reçoit les activations $z_{i,t}^2$ de la deuxième couche cachée. Pour chaque classe $k \in \{0, \dots, 9\}$:

$$o_{i,k}^3 = \sum_{u=1}^{p_2} a_{k,u}^3 z_{i,u}^2 + a_{k,0}^3.$$

On utilise u comme indice muet pour parcourir tous les neurones de la deuxième couche cachée. Le terme $a_{k,u}^3$ est le poids reliant le neurone u de la deuxième couche cachée au neurone représentant la classe k , et $a_{k,0}^3$ est le biais associé à la classe k .

On applique ensuite la fonction softmax :

$$P_{i,k} = \frac{e^{o_{i,k}^3}}{\sum_{\ell=0}^9 e^{o_{i,\ell}^3}}.$$

Pour une image i , la fonction de coût est l'entropie croisée :

$$E_i = - \sum_{\ell=0}^9 y_i^{(\ell)} \ln(P_{i,\ell}).$$

La fonction coût moyenne est :

$$L = \frac{1}{n} \sum_{i=1}^n E_i.$$

Gradient des poids de la couche de sortie

On commence par calculer les gradients de la couche de sortie. Cette partie est très proche de ce que l'on a déjà fait pour le modèle linéaire.

Dans le modèle linéaire, les scores étaient calculés directement à partir des pixels :

$$o_{i,k} = \sum_{m=1}^{784} a_{k,m} x_{i,m} + a_{k,0}.$$

On avait alors obtenu :

$$\frac{\partial L}{\partial a_{k,m}} = -\frac{1}{n} \sum_{i=1}^n x_{i,m} \left(y_i^{(k)} - P_{i,k} \right),$$

ainsi que :

$$\frac{\partial L}{\partial a_{k,0}} = -\frac{1}{n} \sum_{i=1}^n \left(y_i^{(k)} - P_{i,k} \right).$$

Dans notre modèle à deux couches cachées, la couche de sortie a la même structure. La seule différence est qu'elle reçoit les sorties de la deuxième couche cachée $z_{i,t}^2$ au lieu de recevoir directement les pixels $x_{i,m}$.

On a :

$$o_{i,k} = \sum_{t=1}^{p_2} a_{k,t}^3 z_{i,t}^2 + a_{k,0}^3.$$

On reprend donc la formule du modèle linéaire en remplaçant simplement $x_{i,m}$ par $z_{i,t}^2$.

On obtient alors :

$$\frac{\partial L}{\partial a_{k,t}^3} = -\frac{1}{n} \sum_{i=1}^n z_{i,t}^2 \left(y_i^{(k)} - P_{i,k} \right)$$

$$\frac{\partial L}{\partial a_{k,0}^3} = -\frac{1}{n} \sum_{i=1}^n \left(y_i^{(k)} - P_{i,k} \right)$$

Erreur au niveau de la couche de sortie

Comme dans les modèles précédents, on réutilise le résultat obtenu avec la fonction softmax et l'entropie croisée :

$$\frac{\partial E_i}{\partial o_{i,\ell}} = P_{i,\ell} - y_i^{(\ell)}.$$

Ce résultat ne dépend pas du nombre de couches cachées. Il vient seulement du couple softmax et entropie croisée, qui reste identique dans les trois modèles.

Gradient des poids de la deuxième couche cachée

On calcule maintenant le gradient d'un poids de la deuxième couche cachée :

$$\frac{\partial L}{\partial a_{t,q}^2},$$

où t est un neurone fixé de la deuxième couche cachée, et q un neurone fixé de la première couche cachée.

Cette partie se déduit directement du modèle à une couche cachée. Dans ce modèle, on avait obtenu pour un poids de la couche cachée :

$$\frac{\partial E_i}{\partial a_{q,m}^1} = x_{i,m} \phi'(o_{i,q}^1) \left[\sum_{\ell=0}^9 a_{\ell,q}^2 (P_{i,\ell} - y_i^{(\ell)}) \right].$$

Dans le modèle à deux couches cachées, la deuxième couche cachée joue le même rôle que l'unique couche cachée du modèle précédent. La différence est qu'elle ne reçoit plus directement les pixels $x_{i,m}$, mais les activations de la première couche cachée $z_{i,q}^1$.

On effectue donc les remplacements suivants :

$$\begin{aligned} x_{i,m} &\longrightarrow z_{i,q}^1, \\ \phi'(o_{i,q}^1) &\longrightarrow \phi_2'(o_{i,t}^2), \end{aligned}$$

$$a_{\ell,q}^2 \longrightarrow a_{\ell,t}^3.$$

Ainsi, pour une image i , on obtient :

$$\frac{\partial E_i}{\partial a_{t,q}^2} = z_{i,q}^1 \phi_2'(o_{i,t}^2) \left[\sum_{\ell=0}^9 a_{\ell,t}^3 (P_{i,\ell} - y_i^{(\ell)}) \right].$$

Cette formule est bien la même que celle du modèle à une couche cachée, mais adaptée aux nouvelles notations du modèle à deux couches cachées.

En passant au coût moyen :

$$\frac{\partial L}{\partial a_{t,q}^2} = \frac{1}{n} \sum_{i=1}^n \frac{\partial E_i}{\partial a_{t,q}^2}.$$

Donc :

$$\boxed{\frac{\partial L}{\partial a_{t,q}^2} = \frac{1}{n} \sum_{i=1}^n z_{i,q}^1 \phi_2'(o_{i,t}^2) \left[\sum_{\ell=0}^9 a_{\ell,t}^3 (P_{i,\ell} - y_i^{(\ell)}) \right]}$$

Pour le biais $a_{t,0}^2$, le raisonnement est le même que précédemment. Comme un biais peut être vu comme un poids relié à une entrée constante égale à 1, on remplace simplement $z_{i,q}^1$ par 1 :

$$\boxed{\frac{\partial L}{\partial a_{t,0}^2} = \frac{1}{n} \sum_{i=1}^n \phi_2'(o_{i,t}^2) \left[\sum_{\ell=0}^9 a_{\ell,t}^3 (P_{i,\ell} - y_i^{(\ell)}) \right]}$$

Gradient des poids de la première couche cachée

On cherche ici :

$$\frac{\partial L}{\partial a_{q,m}^1},$$

où q est un neurone fixé de la première couche cachée et m un pixel fixé.

Pour la première couche cachée, la couche précédente est la couche d'entrée. On a donc :

$$z_{i,m}^0 = x_{i,m}.$$

La couche suivante est la deuxième couche cachée. On doit donc récupérer l'erreur venant de tous les neurones $t \in \{1, \dots, p_2\}$ de cette deuxième couche.

D'après la formule de rétropropagation du cours, pour une image i , on a :

$$\frac{\partial E_i}{\partial a_{q,m}^1} = x_{i,m} \phi_1'(o_{i,q}^1) \left[\sum_{t=1}^{p_2} a_{t,q}^2 \frac{\partial E_i}{\partial o_{i,t}^2} \right].$$

Dans cette formule, $x_{i,m}$ correspond à l'entrée du poids $a_{q,m}^1$, $\phi_1'(o_{i,q}^1)$ correspond à la dérivée de l'activation du neurone q de la première couche cachée, et la somme représente l'erreur de la deuxième couche cachée ramenée vers le neurone q .

On cherche ensuite le terme $\frac{\partial E_i}{\partial o_{i,t}^2}$. Or, on sait que :

$$o_{i,t}^2 = \sum_{s=1}^{p_1} a_{t,s}^2 z_{i,s}^1 + a_{t,0}^2.$$

Et donc que :

$$\frac{\partial o_{i,t}^2}{\partial a_{t,q}^2} = z_{i,q}^1.$$

D'où :

$$\frac{\partial E_i}{\partial a_{t,q}^2} = \frac{\partial E_i}{\partial o_{i,t}^2} \frac{\partial o_{i,t}^2}{\partial a_{t,q}^2} = \frac{\partial E_i}{\partial o_{i,t}^2} z_{i,q}^1.$$

Or, on sait aussi d'après les calculs sur la deuxième couche cachée que :

$$\frac{\partial E_i}{\partial a_{t,q}^2} = z_{i,q}^1 \phi_2'(o_{i,t}^2) \left[\sum_{\ell=0}^9 a_{\ell,t}^3 (P_{i,\ell} - y_i^{(\ell)}) \right].$$

D'où, par identification :

$$\frac{\partial E_i}{\partial o_{i,t}^2} = \phi_2'(o_{i,t}^2) \left[\sum_{\ell=0}^9 a_{\ell,t}^3 (P_{i,\ell} - y_i^{(\ell)}) \right].$$

On remplace donc ce terme dans la formule précédente :

$$\frac{\partial E_i}{\partial a_{q,m}^1} = x_{i,m} \phi_1'(o_{i,q}^1) \left[\sum_{t=1}^{p_2} a_{t,q}^2 \phi_2'(o_{i,t}^2) \left(\sum_{\ell=0}^9 a_{\ell,t}^3 (P_{i,\ell} - y_i^{(\ell)}) \right) \right].$$

Cette expression donne le gradient pour une seule image i . Pour obtenir le gradient de la fonction coût moyenne, on moyenne ensuite sur toutes les images :

$$\frac{\partial L}{\partial a_{q,m}^1} = \frac{1}{n} \sum_{i=1}^n \frac{\partial E_i}{\partial a_{q,m}^1}.$$

Donc :

$$\boxed{\frac{\partial L}{\partial a_{q,m}^1} = \frac{1}{n} \sum_{i=1}^n x_{i,m} \phi_1'(o_{i,q}^1) \left[\sum_{t=1}^{p_2} a_{t,q}^2 \phi_2'(o_{i,t}^2) \left(\sum_{\ell=0}^9 a_{\ell,t}^3 (P_{i,\ell} - y_i^{(\ell)}) \right) \right]}$$

Pour le biais $a_{q,0}^1$, le raisonnement est le même. La seule différence est que le biais peut être vu comme un poids relié à une entrée constante égale à 1. On remplace donc simplement $x_{i,m}$ par 1 :

$$\boxed{\frac{\partial L}{\partial a_{q,0}^1} = \frac{1}{n} \sum_{i=1}^n \phi_1'(o_{i,q}^1) \left[\sum_{t=1}^{p_2} a_{t,q}^2 \phi_2'(o_{i,t}^2) \left(\sum_{\ell=0}^9 a_{\ell,t}^3 (P_{i,\ell} - y_i^{(\ell)}) \right) \right]}$$

Synthèse de la rétropropagation

Le schéma suivant résume la propagation de l'erreur à travers le réseau :

$$\underbrace{(P_{i,k} - y_i^{(k)})}_{\text{erreur sortie}} \xrightarrow{\times a_{i,t}^3} \underbrace{\delta_{i,t}^2}_{\text{erreur couche 2}} \xrightarrow{\times a_{t,q}^2} \underbrace{\delta_{i,q}^1}_{\text{erreur couche 1}} \xrightarrow{\times x_{i,m}} \nabla_{a_{q,m}^1} L$$

où $\delta_{i,q}^1 = \phi_1'(o_{i,q}^1) \sum_t a_{t,q}^2 \delta_{i,t}^2$. À chaque couche, l'erreur est pondérée par les poids correspondants et filtrée par la dérivée de l'activation, ce qui constitue le principe général de la rétropropagation.

D Analyse complémentaire des erreurs.

D.1 ACP des activations du meilleur modèle

Afin d'avoir une véritable représentation en deux dimensions et de réutiliser nos cours d'Analyse de Données, nous avons voulu faire une ACP sur les activations de la dernière couche cachée du meilleur modèle. Le but est ici d'étudier la représentation apprise par le réseau avant la classification, et non simplement d'étudier les corrélations entre les pixels d'entrée.

Chaque image est ici transformée par le modèle en un vecteur de 128 valeurs, correspondant aux activations des 128 neurones de la couche. Ces valeurs peuvent être vues comme de nouvelles variables, directement utilisées ensuite par la couche de sortie pour prédire la classe.

On peut ensuite représenter cela en deux dimensions grâce à l'ACP. De plus, en colorant les points selon leur vraie classe, on peut voir si le modèle regroupe plutôt bien les images d'un même chiffre. Malgré tout, cette visualisation est assez limitée puisque l'on réduit beaucoup les dimensions et que l'on obtient un nuage de points assez compact. C'est la raison pour laquelle nous avons surtout utilisé la matrice de confusion pour notre analyse.

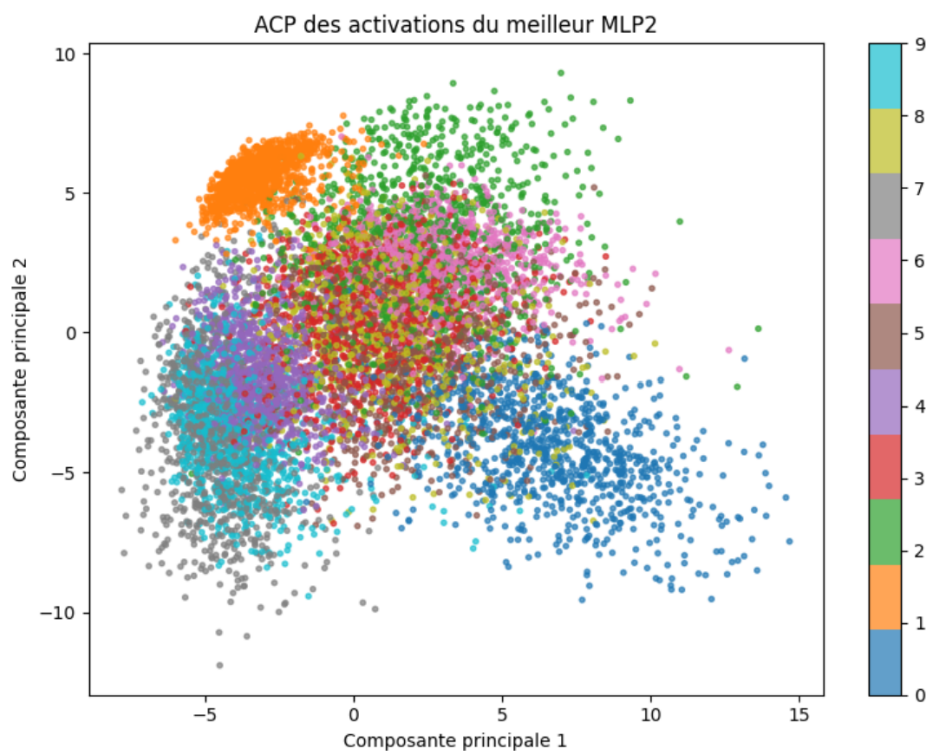


FIGURE 4 – ACP des activations de la dernière couche cachée du meilleur MLP à deux couches.

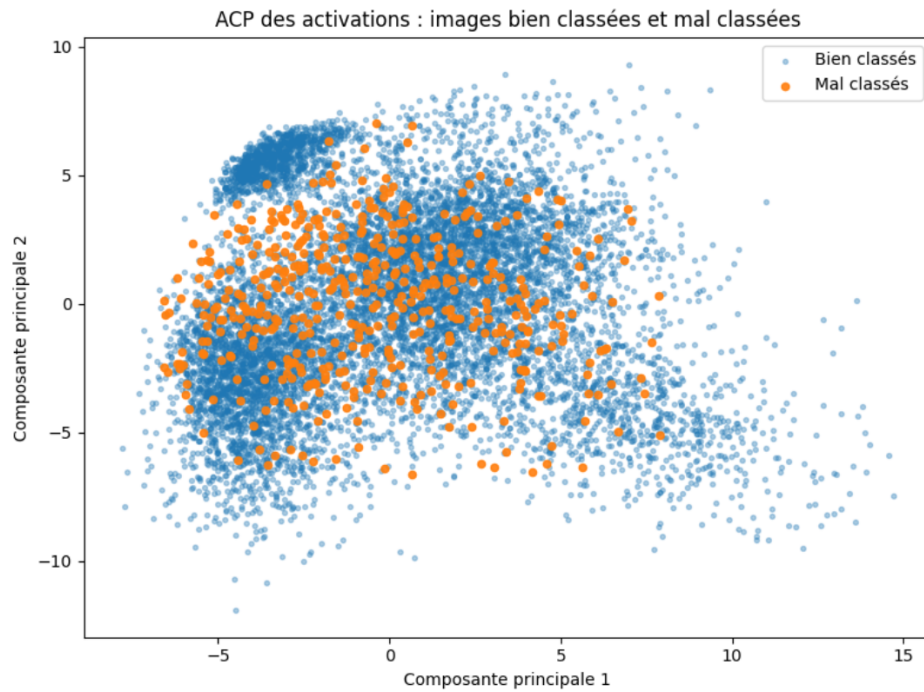


FIGURE 5 – ACP des activations avec les images bien classées et mal classées.

On remarque globalement que les points appartenant à une même classe sont assez bien regroupés entre eux. Cela montre que le réseau arrive à organiser les images selon le chiffre représenté.

Cependant, certaines classes restent très proches les unes des autres. On peut d'ailleurs remarquer que les erreurs sont souvent situées dans ces zones assez confuses. Ces observations ont tendance à confirmer celles faites avec la matrice de confusion, notamment pour les chiffres qui se ressemblent.

D.2 Exemples d'images mal classées

Comparaison entre exemples bien classés et erreurs fréquentes

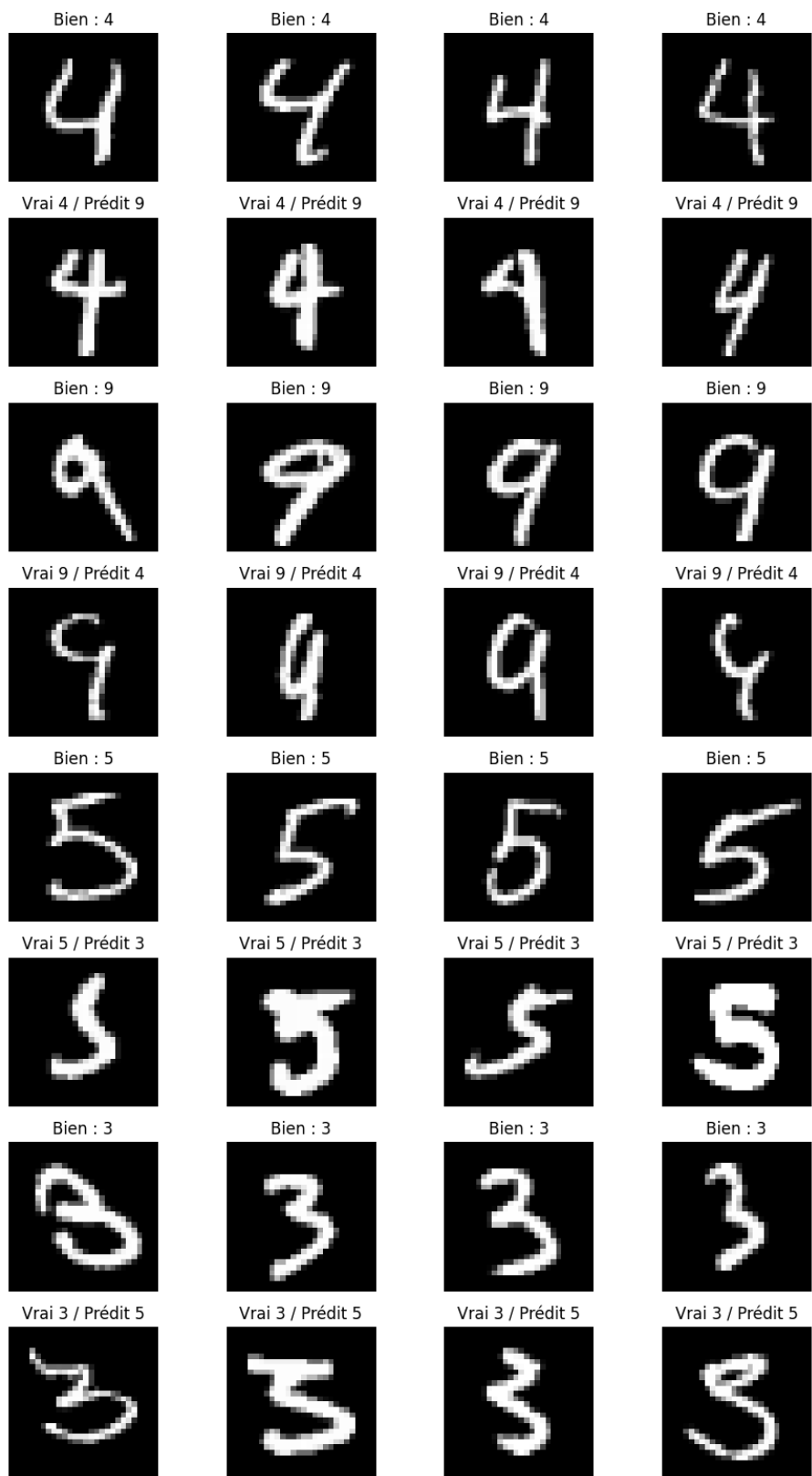


FIGURE 6 – Exemples de confusions courantes.

Exemples aléatoires d'images mal classées

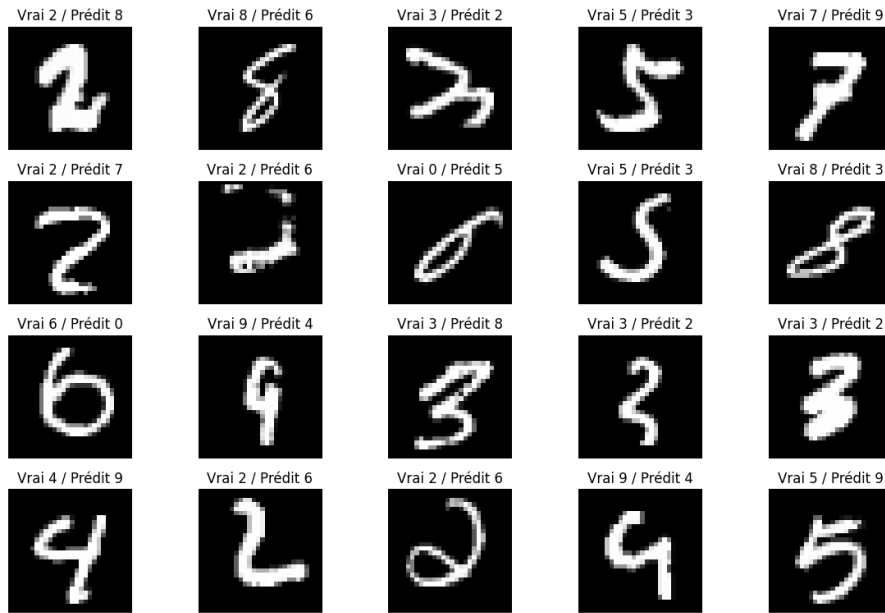


FIGURE 7 – Exemples d'images mal classées choisies aléatoirement.

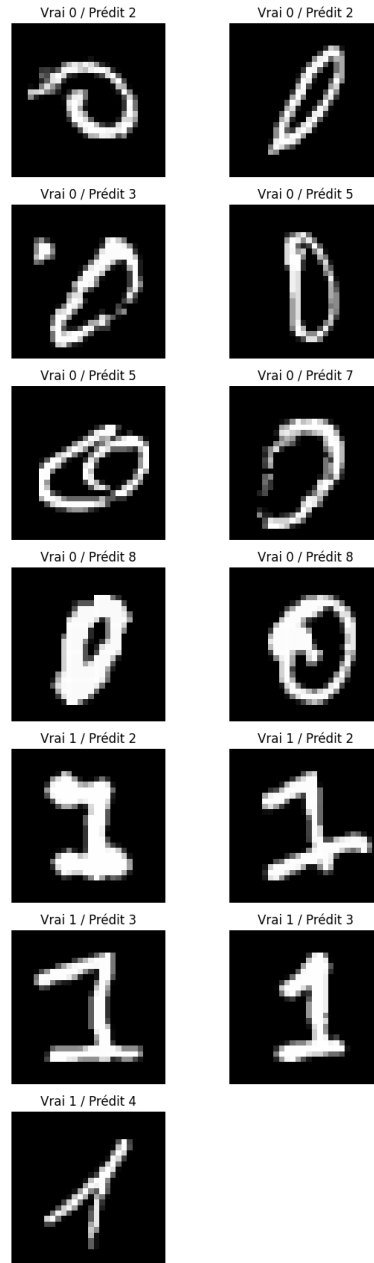


FIGURE 8 – Exemples de confusions plus rares.

E Code du projet et dépôt GitHub

Le code complet du projet est disponible sur notre dépôt GitHub :

https://github.com/bapt4444/projet_Maths_pour_ML.git

Le dépôt est organisé de la manière suivante :

- `linear_model.py` contient l'implémentation du modèle linéaire multi-classe.
- `mlp_model.py` contient l'implémentation des deux modèles MLP, à une et deux couches cachées.
- `utils.py` contient notamment la fonction d'encodage des labels au format one-hot.
- `random_search_utils.py` regroupe les fonctions utilisées pour automatiser la recherche d'hyperparamètres.
- `sample.ipynb` correspond aux premiers essais, notamment l'importation de MNIST et la préparation des données.
- `random_search_experiments.ipynb` contient la recherche d'hyperparamètres, les tests des meilleurs modèles et l'analyse des résultats.

D'autres fichiers secondaires sont également présents dans le dépôt, principalement pour les tests intermédiaires, les affichages et la génération des figures.